

UNITY / URP + HDRP



SQUIRREL FUR SHADER GUIDE

Standalone Shell Fur and Fur Cards demo

Shaders, materials, baked data, runtime integration and performance

CREATED WITH FURBRUSH

The demo runs without the plugin.

FurBrush provides the complete visual authoring workflow

for grooming and baking fur on your own characters.

[FIND FURBRUSH ON THE UNITY ASSET STORE](#)

01 / START HERE

A standalone FurBrush showcase

This package presents a production-ready squirrel groom authored with FurBrush and delivered as ordinary Unity assets. The supplied prefabs use baked, skinned Shell Fur and Fur Cards meshes. They render and animate without FurBrush installed.

The included shaders and materials remain editable, making the package both a character demo and a practical reference for layered fur rendering. FurBrush is only required when you want to paint, scatter, regroom or bake new fur with the full authoring workflow.

AUTHOR YOUR OWN GROOM: [Find FurBrush on the Unity Asset Store](#)

Requirements

- Unity 2022.3 LTS or newer.
- Universal Render Pipeline 14+ for the URP demo.
- High Definition Render Pipeline 14+ for the HDRP demo.
- Use the prefab, scene and materials made for the active render pipeline. URP and HDRP materials are not interchangeable.
- The demo was prepared in Unity 2022.3.62f3 with URP/HDRP 14.0.12.

Quick start

Pipeline	Open	Prefab
URP	Demo/URP_Demo.unity	Prefab/Squirrel_URP.prefab
HDRP	Demo/HDRP_Demo.unity	Prefab/Squirrel_HDRP.prefab

Both prefabs are animated and already contain the final fur renderers. No FurBrush component is required at runtime.

How the squirrel is assembled

Layer	Purpose	Renderer
Body	Skin, eyes and original surface color	Pipeline Lit material
Shell Fur	Dense, continuous short undercoat	Baked skinned shell mesh
Fur Cards	Longer tufts and silhouette breakup	Baked skinned card mesh

Shell Fur fills broad areas efficiently. Fur Cards add the directional shapes that define the cheeks, ears, chest, tail and outer silhouette.

02 / FILES AND MATERIALS

Use the matching pipeline assets

Four local shaders provide the runtime fur look. Keep every file under Fur/Shaders together when moving the demo to another project, because the shader files include the adjacent HLSL support files.

Shader matrix

Shader	Purpose
Squirrel/Fur Shell URP	Short, dense Shell Fur in URP
Squirrel/Fur Shell HDRP	Short, dense Shell Fur in HDRP
Squirrel/Fur Card URP	Baked or authored Fur Cards in URP
Squirrel/Fur Card HDRP	Baked or authored Fur Cards in HDRP

Material map

Material	Role
Materials/Squirrel_URP.mat	Original URP body
Materials/Squirrel_HDRP.mat	Original HDRP body
Materials/Fur_URP_Generated/Squirrel_Shell_URP.mat	Final URP Shell Fur
Materials/Fur_URP_Generated/Squirrel_Cards_URP.mat	Final URP Fur Cards
Materials/Fur_HDRP_Generated/Squirrel_Shell_HDRP.mat	Final HDRP Shell Fur
Materials/Fur_HDRP_Generated/Squirrel_Cards_HDRP.mat	Final HDRP Fur Cards

The directory is named Materials in this package. Duplicate a supplied fur material before experimenting so the original demo look remains available.

Baked runtime data

- The shell mesh carries layer index, original UVs, tangents and skinning data.
- The card mesh carries placement, height, orientation, surface UV and control-map UV data.
- Packed control and width maps are shader data, not ordinary color textures. Preserve their channels and import settings.
- Generic mesh optimization can remove UV or tangent streams required by the fur shaders.

Control map channels

Data	Meaning
Control R	Local Shell Fur length
Control G	Fur coverage / mask
Control B + A	Packed tangent-space comb direction
Width map R	Local Fur Card width

03 / SHELL FUR

Dense short fur from layered geometry

The Shell shader renders the source surface at increasing heights. A procedural strand pattern clips each layer, turning the stack into a continuous short coat that follows the animated character.

Surface input

- Base / Albedo and Base Color define the inherited surface color.
- Use Normal Map adds the original skin normal detail to fur lighting.
- Smoothness controls the broad fiber highlight.
- Use Smoothness Map, Map is Roughness and Smoothness Channel support different texture conventions.

Shape and breakup

Control	What it changes
Max Length	Physical reference length of the coat
Length Scale	Primary global artistic length multiplier
Strand Density	Frequency of the procedural strand pattern; not shell mesh layers
Tip Thinness	Fiber taper toward the tip
Length Jitter / Clumping	Large-scale repetition and grouping
Direction Randomness / Color Variation	Small natural breakup

Color and lighting

- Root Shade and Tip Shade create a vertical color transition.
- Albedo Influence controls inheritance from the squirrel body texture.
- Root AO adds depth close to the skin.
- Aniso Specular shapes directional fiber highlights; Rim / Transmission supports backlight.
- Diffuse Wrap softens diffuse response around the form; Fur Shadow Opacity controls clipped shadows.

Groom and silhouette

- Gravity Strength adds local downward bend.
- Comb Bend Strength, Comb Surface Adherence and Combing Uprightness reproduce the baked groom response.
- View Projection improves apparent continuity through the layered volume.
- Silhouette Correction and Stretch close grazing-angle gaps. Keep Silhouette Tilt restrained.

The supplied shell material and baked mesh are tuned as a pair. If another shell mesh uses a different number of layers, keep the material Shell Count consistent with the mesh.

04 / FUR CARDS

Art-directed tufts over the undercoat

Fur Cards render textured ribbon geometry. In the squirrel groom they provide longer, directional tufts over the short Shell Fur undercoat.

Alpha and fiber shape

Control	Use
Use Alpha Texture	Enables the supplied fur alpha atlas
Alpha from Grayscale	Reads opacity from luminance when alpha is unavailable
Flip Alpha Vertically	Corrects an upside-down strand texture
Alpha Mip Bias	Preserves fine fibers through mip levels; too negative can shimmer
Strands per Card	Procedural strand repetition inside each ribbon
Tip Thinness	Tapers fibers toward the end
Alpha Cutoff	Primary visibility threshold
Alpha Dither Strength	Softens cutoff transitions; too much looks noisy

Shape and motion

- Length Scale changes the global height of the card groom.
- Card Comb Response, Card Lay Flatness and Card Bend shape the baked flow response.
- Gravity Strength adds a secondary downward bend.
- The Width Map provides local card-width variation.
- Billboard To Camera rotates cards toward the view. Keep it disabled for the supplied authored groom.

Albedo projection

The demo cards inherit color from the squirrel surface, preserving body patterns through longer tufts. The baked mesh contains the surface UV data required by this projection. An arbitrary replacement card mesh may not contain compatible UV streams.

Card shader limitation

The included Card shaders are rendering-only. This standalone demo does not contain the FurBrush Scatter Cards tool, card painter or rebaking workflow. Use the full FurBrush plugin to place and edit cards on another character.

05 / RUNTIME SCRIPT

SquirrelFurShellRenderer explained

Scripts/SquirrelFurShellRenderer.cs is an optional, lightweight component for generating temporary Shell layers from a readable MeshRenderer or SkinnedMeshRenderer mesh. The ready demo prefabs use baked final meshes, so this script is not required for their normal playback.

Inspector fields

Field	Behavior
Source Renderer	Optional source. When empty, the Renderer on the same GameObject is used.
Fur Material	Material using Squirrel/Fur Shell URP or Squirrel/Fur Shell HDRP.
Shell Count	Number of generated layers, clamped to 1-32. Geometry cost scales with this value.
Max Fur Length	Expands generated mesh bounds for the expected fur extent. Keep the material Max Length consistent.
Surface Offset	Moves the shell base away from the source surface before shader extrusion.

Setup

- Select a GameObject with a MeshRenderer or SkinnedMeshRenderer.
- Add Squirrel > Fur Shell Renderer.
- Assign a Shell material matching the active render pipeline.
- Enable Read/Write on the source mesh; the component must read vertices and indices on the CPU.
- Set Shell Count, Max Fur Length and optional Surface Offset.
- Use Rebuild Shell Fur from the component context menu when a manual refresh is needed.

Lifecycle

Event	What the component does
OnEnable	Builds the temporary shell mesh and renderer.
OnValidate	Rebuilds after relevant Inspector edits in the Editor.
OnDisable / OnDestroy	Destroys the hidden child and temporary mesh.
Rebuild	Validates source, readability and material; refreshes changed geometry.

The generated child is named `__SquirrelFurShells` and uses `HideAndDontSave`. It is temporary scene state, not a saved production mesh asset.

05A / SCRIPT INTERNALS

Static and skinned mesh handling

The component duplicates the source mesh once for every shell layer. It preserves the data needed by the local shader and creates an appropriate child renderer.

Generated mesh data

- Positions, normals, tangents and UV0 are copied from the source mesh.
- UV1 preserves the source secondary UV layout when available.
- An additional UV stream stores normalized layer height and shell index.
- Every source submesh is repeated for every shell layer.
- The bounds are expanded by Max Fur Length plus Surface Offset.
- Meshes above 65,000 generated vertices automatically use 32-bit indices.

Skinned characters

- Bone weights and bind poses are copied to the generated mesh.
- Bones and root bone are copied from the source SkinnedMeshRenderer.
- The child uses the generated mesh with the assigned fur material.
- Update When Offscreen is enabled to avoid disappearing fur during large animated poses.

Static meshes

- A MeshFilter and MeshRenderer are created on the hidden child.
- Shadow casting, receiving, rendering layer mask and GameObject layer are copied from the source renderer.

Important limitations

- The script does not paint control maps, scatter cards, reduce geometry or create permanent mesh assets.
- It does not synchronize every material property. Match component bounds and material fur length manually.
- Frequent Inspector changes rebuild CPU geometry; avoid using it as a per-frame procedural system.
- The full FurBrush baker is the intended path for optimized, inspectable production hand-off.

Conceptual use

```
var renderer = gameObject.AddComponent<SquirrelFurShellRenderer>();
renderer.sourceRenderer = GetComponent<SkinnedMeshRenderer>();
renderer.furMaterial = shellMaterial;
renderer.shellCount = 16;
renderer.maxFurLength = 0.05f;
renderer.Rebuild();
```

Performance and troubleshooting

Treat the final groom as layered transparent-cutout geometry. Validate it under the intended camera, lighting, animation and target hardware.

Performance priorities

- Shell cost grows approximately with source triangles multiplied by shell count. Reduce layers before reducing texture quality.
- Fur Cards are commonly limited by alpha overdraw. Avoid invisible stacked cards and excessive screen coverage.
- Transparent-cutout fur shadows add cost. Use only necessary shadow distance and lights.
- The supplied fur meshes are skinned and update offscreen to protect animated bounds. Disable this only after validating every pose.
- Profile a player build on target hardware; Scene view numbers are not a shipping benchmark.

Fast diagnosis

Symptom	Check first
Fur is pink	Active pipeline, matching material, all local HLSL files, first shader compiler error
Shell Fur is missing	Shell renderer, material, control G coverage, source mesh Read/Write for runtime generation
Shell layers separate	Final camera distance, layer count, fur length, Silhouette Stretch
Cards disappear / look solid	Alpha Cutoff, alpha texture toggle, luminance mode, Flip Y, mip bias
Fur separates in animation	Correct skinned prefab, bones, root bone, bind poses and extreme poses

Standalone package boundary

Include Assets/Squirrel with its shaders, scripts, materials, textures, meshes, prefabs, animations and matching demo scene. Do not include Assets/FurBrush when distributing this standalone Squirrel demo.

This demo intentionally excludes FurBrush profiles, painting tools, mesh reduction, GPU Strands and editor systems. Its only optional runtime fur authoring component is SquirrelFurShellRenderer.

07 / CREATED WITH FURBRUSH

From a finished demo to your own groom

The Squirrel package contains a final standalone result: shaders, support HLSL, materials, packed maps, baked meshes, prefabs, animation and URP/HDRP scenes. FurBrush provides the authoring workflow that creates and iterates those assets.

The full FurBrush plugin adds

- Scene-view painting for coverage, length, comb direction and width.
- Direct Fur Card scattering and per-card editing.
- Shell Fur, Fur Cards and advanced GPU Strands authoring.
- Independent mixed Shell + Cards layers on one character.
- Reusable profiles and layer-aware material settings.
- Geometry reduction, baking, clearing and final detachment tools.

CREATE THE NEXT GROOM

Use FurBrush to paint, shape and bake production fur directly in the Unity Scene view.

[Find FurBrush on the Unity Asset Store](#)

Support

For questions about the Squirrel demo or FurBrush workflow, contact ripvertices96@gmail.com.